```
{
    "decision": "DENY",
    "determiningPolicies": [
        {
        // policy-managed-device.cedar
            "policyId": "PKtu1UHcdrHYRmmDMEmfzD"
        }
    ],
    "errors": []
}
```

The decision here is `deny`. Although Bob qualifies under the employee view policy, the global forbid policy takes precedence: It denies all employee actions when the device is unmanaged. This demonstrates how ABAC-style global constraints can override more specific relationship- or role-based rules to enforce organizational security requirements.

A.7 Wrapping up the end-to-end example

With these tests, we've observed authorization scenarios for ACME's Customer Collaboration platform in action. Alice, as the owner, can edit the Q3 plan without restrictions. Kate, representing CustCo, can view the document through her customer readers team. Bob, as an ACME employee, can share the document when it is delegatable and he is on a managed device, but the global forbid policy prevents him from viewing it when he connects from an unmanaged laptop.

Taken together, these scenarios show how Cedar policies combine various authorization patterns: ownership baselines, customer sharing, discretionary delegation, and global ABAC-style constraints. Each policy is quite simple on its own, but when combined, they create rich, context-aware authorization behavior. In a real-life deployment, ACME wouldn't be uploading static entities from a file for each request; instead, the Customer Collaboration platform would read them dynamically from its relationship graph.

With schema, policies, entities, and requests in hand, we now have a complete picture of how Cedar can enforce fine-grained, dynamic